

# 电子科技大学

## 实验报告

指导教师：邓健

### 一、实验项目名称：ALU 设计

### 二、实验目的：

掌握采用 Verilog 进行单周期 CPU 设计、仿真和下载的方法。

### 三、实验内容：

分析 MIPS 指令的逻辑功能，定义输入和输出设备对应的内存地址，使用 Verilog 和 ISE 开发工具进行单周期 CPU 逻辑设计、仿真和下载。

### 四、实验原理：

单周期 CPU 的总体模块如图 1 所示。

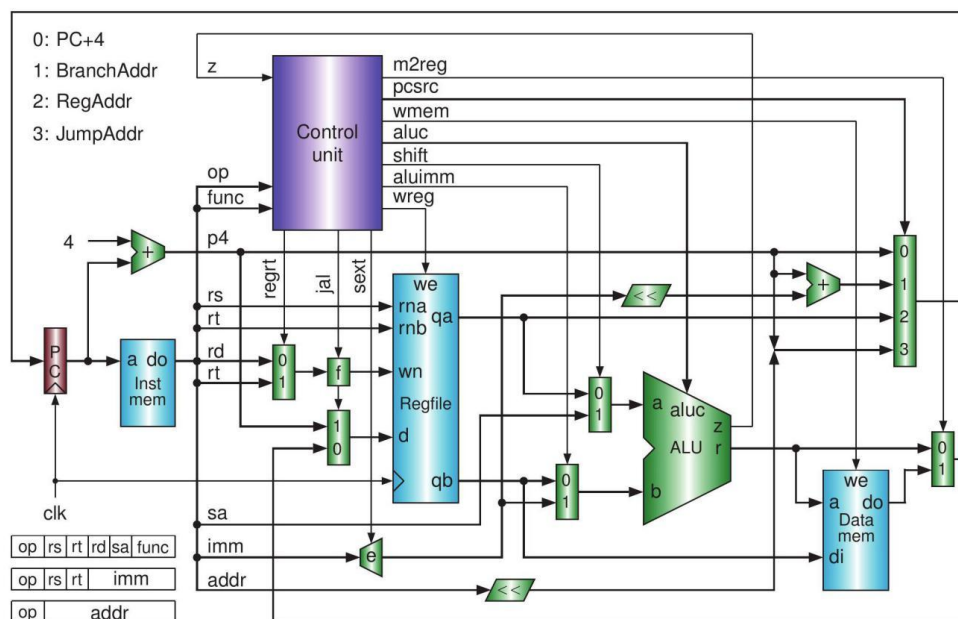


图 1 单周期 CPU + 指令存储器 + 数据存储器的总体模块图

从概念上讲, CPU 部分并不包括存储器 (或主存)。如果把指令存储器和数

据存储器抽出来, 并把 CPU 部分用一个器件符号表示, 则有图 2 所示的结构。我们最好称其为单周期计算机, 因为计算机包括了 CPU 和存储器。

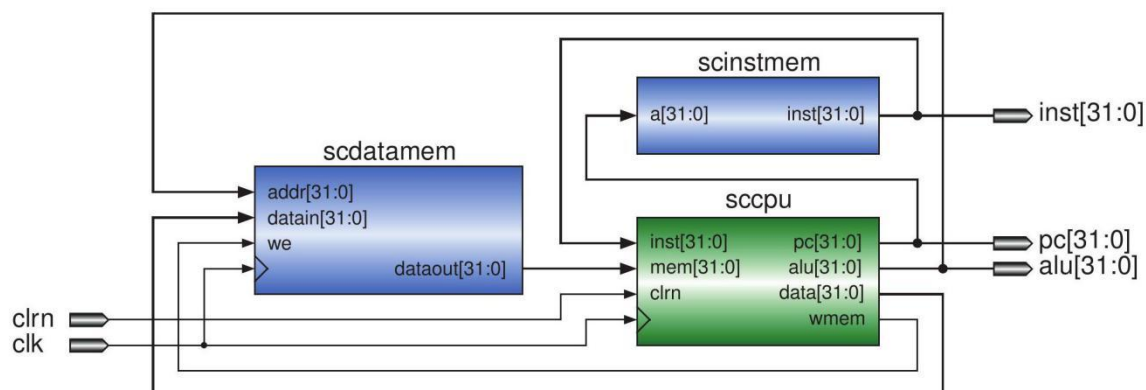


图 2 单周期 CPU + 指令存储器 + 数据存储器的模块图

### 五、实验器材（设备、元器件）:

PC 机、Windows XP、Anvyl 开发板、Xilinx ISE 14.7 开发工具、Digilent Adept 下载工具。

## 六、实验步骤:

在原有工程的基础上增加的指令一共有 subu   addu   slt   sltu   sllv   srlv   srav  
addiu   slti   sltiu.

首先要添加 `subu` 指令，要在 `sccu_dataflow` 里添加相关指令的功能码，查询知道 `subu` 指令的功能码是 `100011`，那么就添加相关的代码，之后分析可知 `subu` 的 `aluc` 控制码是 `0100` 和 `sub` 的一致，同时因为 `r` 型指令牵涉到寄存器的写入，所以 `subu` 的 `wreg` 信号也要置 `1`。因为 `subu` 和 `sub` 指令在 `alu` 的操作是一致的所以不需要 `alu` 中的改变其他的代码。

后续添加的 `addu` 指令和 `subu` 是一致的，同上。

之后是添加 `slt` 指令，`slt` 指令是根据两个寄存器的值的大小比较，来给目的寄存器置 1 或者置 0。依旧是查询 `slt` 的操作码：101010，同时，因为 `slt` 的 `aluc` 控制码和之前的指令没有相似之处，我就直接赋值 1001，因为该指令也牵涉到寄存器的写入，所以 `wreg` 信号也要置 1。在 `alu` 计算过程中，并没有符合 `slt` 的计算，所以要在 `alu` 中添加 `slt` 的计算方式，同时 `alu` 就多了一个信号选项，那么就要扩充 4 选 1 选择器，我直接是按照四位的 `aluc` 来选择的。那么 `slt` 指令就添加成功。

Sltu 指令的添加和 slt 指令类似，不同的地方就是一个是对于无符号数，一个是针对有符号数。那么在 alu 中就要添加新的 sltu 信号用于后续选择，同时 sltu 的赋值语句也是不一样的。同时在选择器也要扩充，那么 sltu 的操作码是 101011，

aluc 控制码我设置的是 1010.

后面的 sllv 指令和 srlv 指令和前面的操作类似，这两条指令的功能是根据源寄存器的低 5 位，给另一个源寄存器进行逻辑左移或者右移，最后把结果赋值给目的寄存器。主要操作是写出操作码和 aluc 码，同时还要再 alu 中加入相关的操作，并且扩充多路选择器。

Srav 指令功能和上面的差不多，但是有一点不同的是，要进行的是算数右移。

后面要添加的是 addiu 指令，可以通过 op 码来直接辨认，所以我们可以通过 op 码的设置，查询可知该指令的 op 码是 001001，同时因为 addiu 的计算过程中牵涉到立即数的扩展，所以他的 sext 信号要置 1，同时 aluimm 也要置 1. 因为 addiu 和 addu 的 alu 计算是一致的，所以，只要把两者的 aluc 控制信号一直就可以了。

Sltihesltiu 指令与前面的 slt、sltu 一致，所以 alu 也不需要多加东西。之后的操作和 addiu 一致。

## 七、关键源代码：

### 操作码（功能码）：

```
wire i_subu = r_type & func[5] & ~func[4] & ~func[3] & ~func[2] &
func[1] & func[0] ;

wire i_addu = r_type & func[5] & ~func[4] & ~func[3] & ~func[2] &
~func[1] & func[0] ;

wire i_slt = r_type & func[5] & ~func[4] & func[3] & ~func[2] & func[1]
& ~func[0] ;

wire i_sltu = r_type & func[5] & ~func[4] & func[3] & ~func[2] & func[1]
& func[0] ;

wire i_sllv = r_type & ~func[5] & ~func[4] & ~func[3] & func[2] &
~func[1] & ~func[0] ;

wire i_srlv = r_type & ~func[5] & ~func[4] & ~func[3] & func[2] &
func[1] & ~func[0] ;

wire i_srav = r_type & ~func[5] & ~func[4] & ~func[3] & func[2] &
func[1] & func[0] ;
wire i_addiu = ~op[5] & ~op[4] & op[3] & ~op[2] & ~op[1] & op[0] ;

wire i_slti = ~op[5] & ~op[4] & op[3] & ~op[2] & op[1] & ~op[0] ;

wire i_sltiu = ~op[5] & ~op[4] & op[3] & ~op[2] & op[1] & op[0] ;
```

## Aluc 码:

```
assign aluc[3] = i_sra | i_slt | i_sltu | i_sllv | i_srlv | i_srav | i_slti | i_sltiu;
assign aluc[2] = i_sub | i_or | i_srl | i_sra | i_ori | i_lui | i_subu | i_srlv | i_srav;
assign aluc[1] = i_xor | i_sll | i_srl | i_sra | i_xori | i_beq | i_bne | i_lui | i_sltu | i_sllv | i_srav | i_sltiu;
assign aluc[0] = i_and | i_or | i_sll | i_srl | i_sra | i_andi | i_ori | i_slt | i_sllv | i_srav | i_slti;
```

## wreg 信号:

```
assign wreg = i_add | i_sub | i_and | i_or | i_xor | i_sll | i_srl | i_sra |
              i_addi | i_andi | i_ori | i_xori | i_lw | i_lui | i_jal | i_subu |
              i_addu | i_slt | i_sltu | i_sllv | i_srlv | i_srav | i_addiu | i_slti |
              i_sltiu;
```

## 有关 i 型指令的信号:

```
assign regrt = i_addi | i_andi | i_ori | i_xori | i_lw | i_lui | i_addiu | i_slti | i_sltiu;
assign jal = i_jal;
assign m2reg = i_lw;
assign shift = i_sll | i_srl | i_sra;
assign aluimm = i_addi | i_andi | i_ori | i_xori | i_lw | i_lui | i_sw | i_addiu | i_slti | i_sltiu;
assign sext = i_addi | i_lw | i_sw | i_beq | i_bne | i_addiu | i_slti | i_sltiu;
```

## Alu:

```
// page 124
module alu (a,b,aluc,r, z);
  input [31:0] a, b;
  input [3:0] aluc;
  output [31:0] r;
  output z;

  wire [31:0] d_and = a & b;
  wire [31:0] d_or = a | b;
  wire [31:0] d_xor = a ^ b;
  wire [31:0] d_lui = {b[15:0], 16'h0};
  wire [31:0] d_and_or = aluc[2]? d_or : d_and;
  wire [31:0] d_xor_lui = aluc[2]? d_lui : d_xor;

  wire [31:0] d_as, d_sh;
  wire [31:0] d_slt = ($signed(a) < $signed(b)) ? 32'b1 : 32'b0;
  wire [31:0] d_sltu = (a < b) ? 32'b1 : 32'b0;
  wire [31:0] d_sllv = b << (a[4:0]);
  wire [31:0] d_srlv = b >> (a[4:0]);
  wire [31:0] d_srav = $signed(b) >>> (a[4:0]);

  addsub32 as32 (a,b,aluc[2], d_as);
  shift shifter (b, a[4:0], aluc[2], aluc[3], d_sh);
  mux9 select (d_as, d_and_or, d_xor_lui, d_sh, d_slt, d_sltu, d_sllv, d_srlv, d_srav, aluc, r);

  assign z = ~|r;

endmodule
```

## 多路选择器:

```
// page 40
module mux9(a0, a1, a2, a3, a4, a5, a6, a7, a8, s, y);
    input [31:0] a0, a1, a2, a3, a4, a5, a6, a7, a8;
    input [3:0] s;
    output [31:0] y;

    function [31:0] select;
        input [31:0] a0, a1, a2, a3, a4, a5, a6, a7, a8;
        input [3:0] s;
        case (s)
            4'b0000 : select = a0;
            4'b0100 : select = a0;
            4'b1000 : select = a0;
            4'b0001 : select = a1;
            4'b0101 : select = a1;
            4'b1101 : select = a1;
            4'b0010 : select = a2;
            4'b0110 : select = a2;
            4'b1110 : select = a2;
            4'b0011 : select = a3;
            4'b0111 : select = a3;
            4'b1001 : select = a4;
            4'b1010 : select = a5;
            4'b1011 : select = a6;
            4'b1100 : select = a7;
            4'b1111 : select = a8;
        endcase
    endfunction

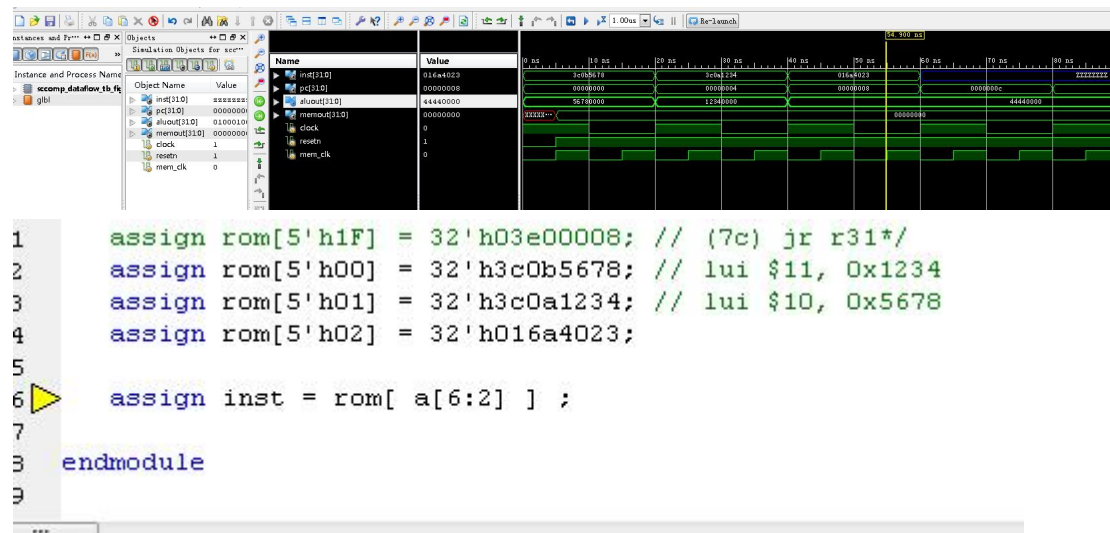
    assign y = select(a0, a1, a2, a3, a4, a5, a6, a7, a8, s);

endmodule
```

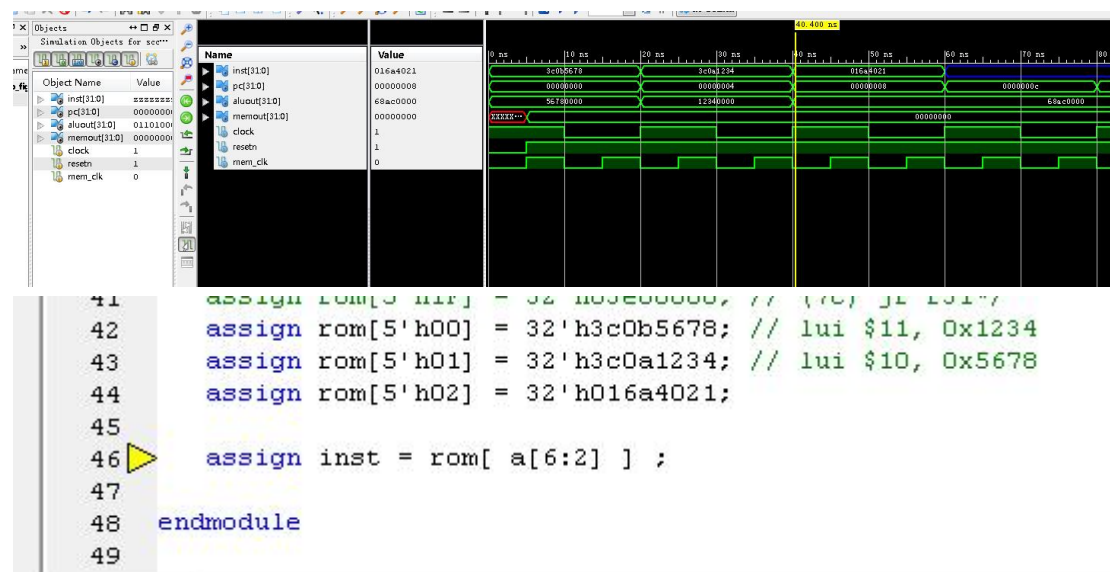
## 八、实验结论：

添加的指令都运行成功。

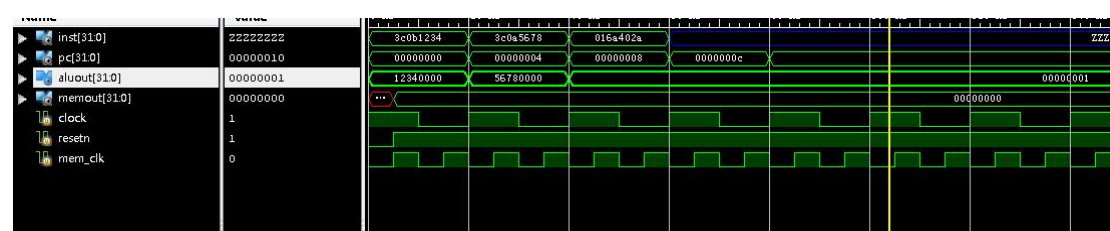
Subu:



Addu 指令:



Slt:



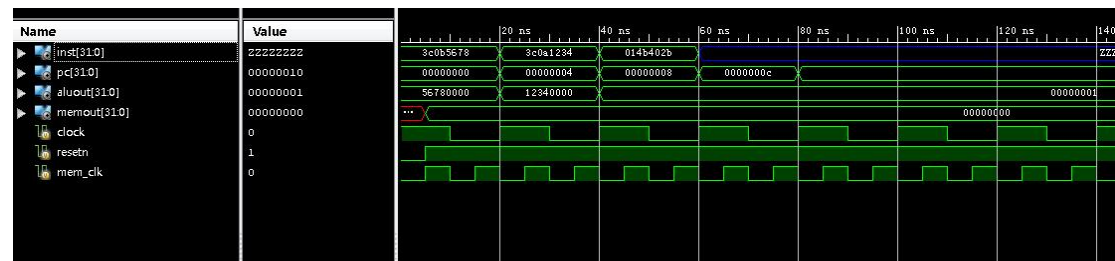


```

41  assign rom[5'h02] = 32'h016a402a; // lui $10, 0x16a402a
42  assign rom[5'h00] = 32'h3c0b1234; // lui $11, 0x1234
43  assign rom[5'h01] = 32'h3c0a5678; // lui $10, 0x5678
44  assign rom[5'h02] = 32'h016a402a; |
45
46  assign inst = rom[ a[6:2] ] ;
47
48  endmodule
49

```

Sltu:



```

41  assign rom[5'h00] = 32'h03e00000; // ori $11, $0, 0x0003
42  assign rom[5'h00] = 32'h3c0b5678; // lui $11, 0x1234
43  assign rom[5'h01] = 32'h3c0a1234; // lui $10, 0x5678
44  assign rom[5'h02] = 32'h014b402b;
45
46  assign inst = rom[ a[6:2] ] ;
47
48  endmodule
49

```

Sllv:

```

assign rom[5'h00] = 32'h340B0003; // ori $11, $0, 0x0003
assign rom[5'h01] = 32'h340A000F; // ori $10, $0, 0x000F
assign rom[5'h02] = 32'h016A4004; // sllv $8, $10, $11

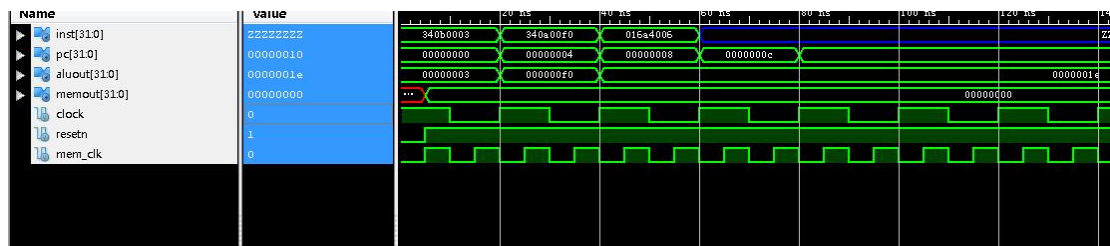
assign inst = rom[ a[6:2] ] ;

endmodule

```



Srlv:

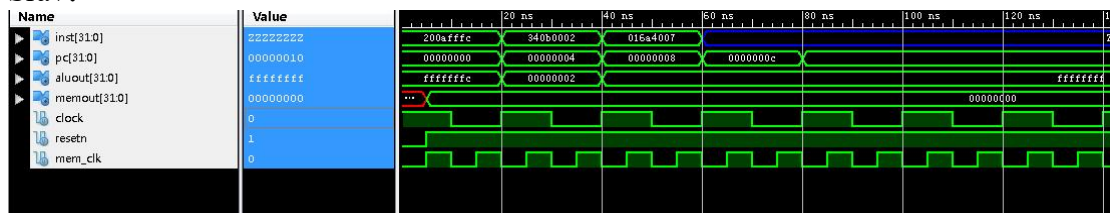


```
//srlv
assign rom[5'h00] = 32'h340B0003; // ori $11, $0, 0x0003
assign rom[5'h01] = 32'h340A00f0; // ori $10, $0, 0x000F
assign rom[5'h02] = 32'h016A4006; // srlv $8, $10, $11

assign inst = rom[ a[6:2] ] ;

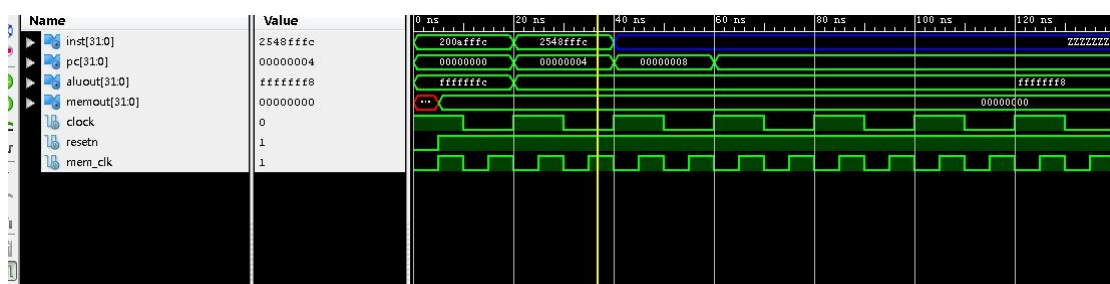
endmodule
```

Srav:



```
55 assign rom[5'h00] = 32'h200afffc; //addi r10 r0 -4
56
57 assign rom[5'h01] = 32'h340B0002; // ori $11, $0, 0x0002
58 assign rom[5'h02] = 32'h016A4007; // srav $8, $10, $11
59
60
61
```

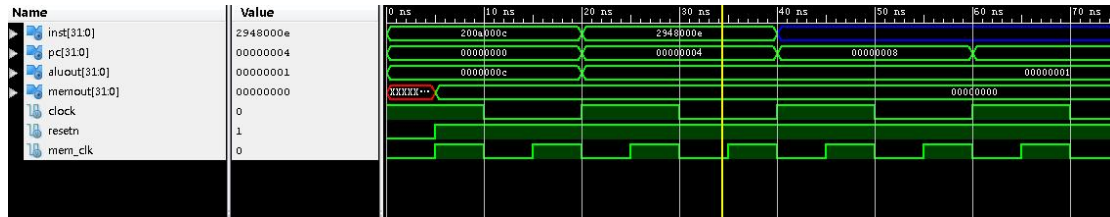
Addiu:



```
assign rom[5'h00] = 32'h200afffc; //addi r10 r0 -4
assign rom[5'h01] = 32'h2548ffff; //addiu r8 r10 -4;
```

Slti:



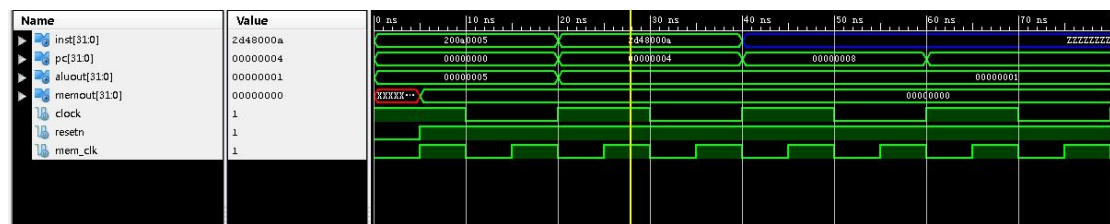


```

64 //slti
65 assign rom[5'h00] = 32'h200A000C; // addi $10, $0, c
66 assign rom[5'h01] = 32'h2948000E; // slti $8, $10, -2
67
68

```

Sltiu:



```

//sltiu
assign rom[5'h00] = 32'h200A0005; // addi $10, $0, 5
assign rom[5'h01] = 32'h2D48000A; // sltiu $8, $10, 10

```

## 九、总结及心得体会：

深入了解了单周期 cpu 指令的形成与原理。

## 十、对本实验过程及方法、手段的改进建议：

无

报告评分：

指导教师签字：